

1 Mobile CarCare: Application Working Guide

Repository: [Abdobeaih/Car-Wash](https://github.com/Abdobeaih/Car-Wash)

Author: Manus AI

Review basis: Complete tracked source tree and configuration reviewed on 18 August 2026.

1.1 1. Purpose and system overview

Mobile CarCare is a full-stack application for booking mobile car-wash and detailing services. A customer chooses one or more services, selects a vehicle, adds optional extras, supplies the service location, chooses a future date and time, reviews the price, and submits a booking. The provider can then manage services, add-ons, customers, bookings, messages, notifications, and the calendar through an admin area.

The project is a workspace monorepo. The web application is built with Next.js App Router, React, TypeScript, and Tailwind CSS. The API is built with NestJS and TypeScript, and MongoDB accessed through Mongoose is the source of truth for users, vehicles, catalog data, bookings, contact messages, and notifications.

The most important business rule is that the backend—not the browser—recalculates service duration, add-on prices, total price, end time, and booking conflicts before saving a booking.

1.2 2. Repository structure

Path	Responsibility
apps/web	Next.js frontend, public pages, customer dashboard, admin dashboard, shared UI, auth state, and API proxy.
apps/api	NestJS REST API, MongoDB schemas, validation, JWT auth, business rules, notifications, seed data, and Vercel handler.
package.json	Root workspace scripts for development, build, lint, type checking, and tests.
.env.example and workspace env examples	Local and deployment configuration templates.
apps/web/public/images/services	Service illustrations and homepage imagery.

The main API modules are auth, users, services, addons, vehicles, bookings, admin, notifications, contact, seed, and health. The frontend route tree includes public pages, /book, /dashboard/*, /admin/*, and the dynamic /api/[...path] proxy.

1.3 3. How the application runs

At startup, Nest loads configuration through ConfigModule, connects to MongoDB, enables CORS, and installs a global ValidationPipe. In development, MongoDB defaults to mongodb://127.0.0.1:27017/mobile-car-care if DATABASE_URL is not supplied. In production, the API fails fast when DATABASE_URL is missing. The API listens on PORT, defaulting to port 3001.

The browser calls the frontend's `/api` path. `apps/web/src/lib/api.ts` selects `/api` in the browser and uses `API_URL`, `NEXT_PUBLIC_API_URL`, or `http://localhost:3001` during server rendering. The Next.js catch-all route forwards the HTTP method, query string, headers, and request body to the API. If the upstream API cannot be reached, the proxy returns HTTP 502 with a message directing the operator to check `API_URL`.

The API can also be deployed to Vercel. `apps/api/serverless.ts` creates the Nest application once, initializes it, caches the HTTP adapter, and forwards subsequent Vercel requests to that adapter. The API's `vercel.json` maps all paths to this handler.

1.4 4. Local setup

Install Node.js 20.9 or later, npm, MongoDB, and Git. From the repository root, run:

```
npm install
cp .env.example .env
cp apps/api/.env.example apps/api/.env
cp apps/web/.env.example apps/web/.env.local
```

Set at least the API values below. The local MongoDB URI is suitable for development only.

Variable	File	Purpose
DATABASE_URL	apps/api/.env	MongoDB connection string.
JWT_SECRET	apps/api/.env	Secret used to sign and validate JWTs. Use a long random value outside development.
JWT_EXPIRES_IN	apps/api/.env	Token lifetime, such as 7d.
PORT	apps/api/.env	API port, normally 3001.
CORS_ORIGINS	apps/api/.env	Comma-separated allowed production frontend origins.
NEXT_PUBLIC_API_URL	apps/web/.env.local	Local API base URL, normally <code>http://localhost:3001</code> .
API_URL	Web hosting environment	Server-side API URL used by the production Next.js proxy.

Run the two applications in separate terminals:

```
npm run dev:api # API at http://localhost:3001
npm run dev:web # Web at http://localhost:3000
```

On the first API start, the seed module creates demo catalog records and users when appropriate. Development credentials are printed to the API console. Do not reuse demo credentials or a development JWT secret in production.

1.5 5. Customer workflow

The customer begins on the public home or services page. Public service data comes from GET /services, which returns active services sorted by base price. A service detail page uses GET /services/:slug. Public add-ons are available through GET /add-ons.

Registration uses POST /auth/register. The API validates name, email, and an eight-character minimum password, hashes the password with bcrypt, creates a customer role, and returns a JWT plus the public user profile. Login uses POST /auth/login; the frontend stores the token in browser localStorage under mcc_token. The auth context restores the session by calling GET /auth/me. Logout calls POST /auth/logout and then clears the local token; the API itself is stateless and does not revoke an issued token.

The booking page implements this sequence:

Step	What happens
Service	The customer chooses one or more active services.
Vehicle	The customer selects a vehicle belonging to the signed-in account.
Add-ons	Optional active add-ons are attached to each service selection.
Location	Country, city, address, optional coordinates, and notes are collected.
Date	A non-past date is selected.
Time	The frontend requests availability for the selected date and service IDs.
Review	The customer sees service lines, duration, location, appointment time, and an estimated total.
Confirm	POST /bookings sends the complete selection to the backend for authoritative validation and pricing.

The backend verifies vehicle ownership, service and add-on existence, and active status. It recalculates each line item using catalog prices, sums duration, computes the end time, enforces the 09:00–18:00 working window, and rejects an overlap with any non-cancelled booking using HTTP 409. A successful booking is stored as PENDING with payment status PENDING. Admin users receive an in-app notification for a new booking.

Customers can view their bookings with GET /bookings, open a booking with GET /bookings/:id, and cancel with POST /bookings/:id/cancel. Cancellation is limited to the customer's own booking and is rejected for completed or already-cancelled bookings. A cancellation frees the time interval because conflict checks ignore cancelled bookings, and administrators receive a notification.

1.6 6. Admin workflow

Every admin endpoint is protected by both `JwtAuthGuard` and `RolesGuard`; the required role is `ADMIN`. The frontend's `RequireRole` guard provides the corresponding user experience, but the API remains the security boundary.

The admin dashboard calls `GET /admin/dashboard` to calculate booking counts, customer count, and revenue from non-cancelled bookings. Admins can create, edit, activate, deactivate, and delete catalog services and add-ons. Service slugs are generated from names when not supplied. The booking list supports status and customer-name/email search, while booking detail allows status changes through `PATCH /admin/bookings/:id/status`. A changed status generates a customer notification.

The calendar endpoint, `GET /admin/calendar`, returns non-cancelled bookings ordered by date and start time, with customer, vehicle, and service data populated. The customers page lists customer accounts and booking counts. Contact messages arrive through public `POST /contact`; the backend stores them, notifies all admins, and exposes them through admin message endpoints. Admins can read the unread count and mark messages as read. Admin and customer notification pages use the authenticated notifications endpoints to list notifications, mark one read, or mark all read.

1.7 7. API route reference

Method	Route	Access	Purpose
GET	/health	Public	API health check.
POST	/auth/register	Public	Create a customer and issue JWT.
POST	/auth/login	Public	Authenticate and issue JWT.
POST	/auth/logout	Authenticated	Return logout acknowledgement.
GET / PATCH	/auth/me	Authenticated	Read or update the current profile.
POST	/auth/change-password	Authenticated	Change password after checking current password.
POST	/auth/forgot-password	Public	Generate a one-hour reset token.
POST	/auth/reset-password	Public	Reset password with the token.
GET	/services and /services/:slug	Public	Browse active services.
GET	/add-ons	Public	Browse active add-ons.

Method	Route	Access	Purpose
GET / POST / PATCH / DELETE	/vehicles and / vehicles/:id	Customer	Manage owned vehicles.
GET	/availability? date=YYYY-MM-DD&serviceIds=...	Public	Return available hourly slots.
GET / POST	/bookings	Customer	List or create bookings.
GET	/bookings/:id	Customer	Read one owned booking.
POST	/bookings/:id/cancel	Customer	Cancel one owned booking.
GET / POST / PATCH / DELETE	/admin/services* and /admin/add-ons*	Admin	Manage catalog records.
GET / PATCH	/admin/bookings*	Admin	Review and update bookings.
GET	/admin/dashboard, / admin/calendar, / admin/customers	Admin	Dashboard, calendar, and customer data.
GET / PATCH	/notifications*	Authenticated	Read and update in-app notifications.
POST	/contact	Public	Submit a contact message.

1.8 8. Data model

Model	Important fields	Relationship or rule
User	Name, lowercase email, bcrypt password, role, reset-token fields	A user is a CUSTOMER or ADMIN; email is used for login.
Vehicle	User ID, brand, model, year, plate number, color	A customer may access only vehicles whose userId matches the JWT subject.
CarService	Name, slug, description, base price, duration, image URL, active flag	Inactive services cannot be booked.

Model	Important fields	Relationship or rule
AddOn	Name, description, price, active flag	Inactive add-ons cannot be booked.
Booking	Customer, vehicle, services, add-ons, date/time, duration, subtotal, total, status, payment status, location	Booking records preserve calculated duration and price snapshots.
Notification	Recipient, type, title, message, data, read flag	Notifications are scoped to the recipient.
ContactMessage	Name, email, message, read flag	New messages notify every admin.

The booking document keeps both a `services` array for multi-service line items and top-level `serviceId/addOnIds` fields that mirror the first service selection. This supports the current display code and compatibility with the older single-service shape.

1.9 9. Security and validation behavior

All request bodies pass through class-validator rules. Unknown properties are rejected, IDs are checked with a MongoDB ObjectId pipe, and the JWT strategy reloads the user from MongoDB so a deleted account cannot continue using the token. Admin authorization is enforced server-side. Password reset tokens are stored as SHA-256 hashes and expire after one hour.

The current implementation should still be hardened before a production launch. Password reset codes are returned directly because there is no email provider, so the feature is suitable for development but not real customer recovery. Logout clears the browser token but does not revoke the JWT server-side. Booking conflicts are checked with a read-then-write query; a database transaction or stronger concurrency strategy would be advisable for high-volume scheduling. Payment status is modeled but no payment gateway is connected.

1.10 10. Verification performed

The repository was installed and validated from the source tree. The production build succeeded for both workspaces, and the API test suite passed with two suites and six tests. The root test command still exits non-zero because the web workspace has no test script. Type checking reports missing module declarations for four binary imports in `apps/web/src/app/page.tsx`, even though the corresponding image files are present in the working tree; the imports should be changed to public URL strings or provided with appropriate Next.js image declarations. npm installation also reported ten audit findings, including three moderate and seven high-severity advisories, which should be reviewed before deployment.

The exact validation commands are:

```
npm run build
npm run typecheck
npm test
```

A successful build is therefore not the same as a clean quality gate: build currently passes, API tests pass, while the aggregate typecheck and aggregate test commands require follow-up fixes.

1.11 11. Deployment checklist

For the API deployment, provide DATABASE_URL, a strong JWT_SECRET, JWT_EXPIRES_IN, PORT if required by the host, and production CORS_ORIGINS. For the web deployment, set API_URL to the deployed API URL so the same-origin proxy can forward browser requests. Configure the production database, verify the /health route, and confirm that the web origin is allowed by the API CORS configuration.

Before accepting real bookings, fix the homepage image import/typecheck issue, add a web test script or change the root test strategy, review npm audit findings, replace the reset-token response with email delivery, integrate payment processing, and add concurrency protection around booking creation. Finally, replace all seed credentials and development secrets.

1.12 References